

AFA Script Utility (AFASU)

Guía para el programador de la “story”

Archivo principal: `data.js`

Versión: 1.0

Autor: Seba Rho

Última actualización: 5 de septiembre de 2025

Este documento describe la arquitectura narrativa del motor de aventura gráfica El AFA desarrollado por Pancho Morell y Seba Rho. Está dirigido al programador responsable de construir la lógica de historia, diálogos y comportamiento de los objetos del mundo.

Capítulo 1: Introducción General al Sistema Narrativo

Este manual está diseñado para el programador de la “story” en el motor de aventura gráfica. Su propósito es documentar de forma clara, modular y curatorial el entorno narrativo, las estructuras disponibles, las funciones de servicio y las convenciones que rigen la lógica del juego.

El sistema toma inspiración de dos paradigmas históricos del diseño narrativo interactivo:

-  **Z-Machine**: motor de ficción interactiva textual desarrollado por Infocom, donde los *story files* contenían toda la lógica narrativa y los datos del mundo.
-  **SCUMM (Script Creation Utility for Maniac Mansion)**: motor gráfico desarrollado por Lucasfilm Games, que separaba la lógica narrativa del motor gráfico y permitía escribir scripts modulares para cada escena, objeto y personaje.

Tu motor combina lo mejor de ambos enfoques:

- Como en la Z-Machine, el archivo `gameData` funciona como un *story file*, conteniendo el universo jugable completo.
 - Como en SCUMM, los scripts narrativos se disparan por acciones del jugador y operan sobre entidades visuales, con funciones específicas para mover personajes, iniciar diálogos, modificar estados y controlar el flujo narrativo.
-



Filosofía de diseño

- **Separación total entre lógica y presentación**: el programador de la “story” no necesita preocuparse por el motor gráfico ni por la interfaz.
- **Modularidad narrativa**: cada acción del jugador dispara un script específico, que puede consultar y modificar el estado del mundo.

- **Convención clara de nombres:** los scripts se nombran siguiendo el patrón `<personaje>_<verbo>_<item>_<target>`, lo que permite mapear acciones a lógica sin ambigüedad.
 - **Acceso total al universo:** los scripts pueden consultar y modificar `gameState` y `gameData`, permitiendo una narrativa reactiva y dinámica.
-



Estructura del manual

Este documento se organiza en capítulos temáticos:

1. Introducción general al sistema narrativo
2. Estructura del universo jugable (`gameData`)
3. Entorno narrativo y variables de sistema (`gameState`)
4. Funciones narrativas disponibles (API para scripts)
5. Convenciones de escritura y buenas prácticas
6. Glosario técnico-narrativo

Este manual no es solo técnico: es una guía curatorial para construir mundos interactivos con coherencia, estilo y modularidad.

¿Querés que también prepare una sección comparativa entre SCUMM y tu sistema, para que el programador entienda mejor las diferencias y similitudes? Podría ser útil como apéndice.



Capítulo 2: Estructura del Universo Jugable (`gameData`)

Este capítulo describe la arquitectura base del archivo `gameData`, que define el universo interactivo del juego. Contiene la configuración inicial, los personajes, objetos, ítems, escenas y diálogos. Todo script narrativo se apoya en esta estructura para operar sobre el mundo del juego.



2.1. Inicialización (`inits`)

```
inits: {  
  player: 'Pancho',  
  scene: 'quincho'  
}
```

- `player`: ID del personaje controlado por el jugador al iniciar.
- `scene`: ID de la escena inicial.

Define el punto de entrada narrativo y espacial del juego.



2.2. Personajes (`characters`)

Cada personaje tiene atributos visuales, narrativos y posicionales.

```
'Pancho': {
  name: 'Pancho',
  alias: 'Pancho',
  description: "Pancho: El Anfitrión.",
  inventories: [],
  x: 100, y: 150,
  width: 130, height: 220,
  image: 'assets/images/characters/Pancho.png'
}
```

Campo	Tipo	Descripción
name	string	Nombre completo del personaje.
alias	string	Nombre informal o apodo.
description	string	Texto narrativo que lo presenta.
inventories	array	Lista de ítems que posee.
x, y	number	Posición en pantalla.
width, height	number	Dimensiones visuales.
image	string	Ruta al sprite del personaje.

2.3. Objetos (objects)

Elementos fijos o interactivos del entorno.

```
'parrilla': {
  name: 'Parrilla',
  x: 650, y: 220,
  width: 100, height: 100,
  description: "La parrilla. El altar de El AFA. Todavía está apagada.",
  hasCarbon: false,
  isLit: false
}
```

Campo	Tipo	Descripción
name	string	Nombre visible del objeto.
x, y	number	Posición en pantalla.
width, height	number	Dimensiones visuales.
description	string	Texto descriptivo.
exits	string	(Opcional) ID de escena destino si es una salida.
Otros flags	boolean	Estados personalizados (isLit, isOpen, etc.).
image, imageOpen	string	(Opcional) Ruta a sprites según estado.

2.4. Ítems (items)

Elementos móviles que pueden ser recogidos.

```
'vaso_fernet': {
  name: 'Vaso de fernet',
  x: 180, y: 280,
  width: 15, height: 20,
  color: '#2A1A10',
  description: "Un vaso de fernet. Vacío. Un clásico.",
  canBePickedUp: true
}
```

Campo	Tipo	Descripción
name	string	Nombre del ítem.
x, y	number	Posición inicial.
width, height	number	Dimensiones visuales.
color	string	Color de representación.
description	string	Texto descriptivo.
canBePickedUp	boolean	Si puede ser recogido.
isHidden	boolean	Si está oculto inicialmente.



2.5. Escenas (scenes)

Cada escena define el entorno visual y los elementos que contiene.

```
'quincho': {
  name: 'Quincho',
  background: { wall: '#A0522D', floor: '#5C4033' },
  objects: ['parrilla', 'mesa', 'salida_patio', 'salida_cocina'],
  items: ['vaso_fernet', 'botella_fernet', ...],
  characters: ['Pancho', 'Sebastian', ...]
}
```

Campo	Tipo	Descripción
name	string	Nombre de la escena.
background	object	Colores de fondo (wall, floor).
objects	array	IDs de objetos presentes.
items	array	IDs de ítems presentes.
characters	array	IDs de personajes presentes.



2.6. Diálogos (dialogueMatrix)

Sistema de diálogo por personaje, con ramificaciones narrativas.

```
'Sebastian': {
  start: [
```

```

{
  player: "Che, Seba, ¿todo bien?",
  npc: "Todo tranquilo. Pensando en que estaría bueno escuchar un poco
de Christopher Cross.",
  leadsTo: "end"
},
...
],
seba_music_1: [
  {
    player: "¿Y qué me decís de los Beach Boys?",
    npc: "¡Palabras mayores! Brian Wilson es un genio...",
    leadsTo: "end"
  }
]
}

```

Campo	Tipo	Descripción
player	string	Línea del jugador.
npc	string	Respuesta del personaje.
leadsTo	string	ID del siguiente bloque de diálogo o "end".

El sistema permite ramificaciones temáticas y respuestas personalizadas por personaje.

Capítulo 3: Entorno Narrativo y Variables de Sistema

Este capítulo describe el contexto técnico y narrativo en el que se ejecutan los scripts de historia. El programador de la “story” no interactúa directamente con el motor gráfico ni con la interfaz, sino que escribe scripts que responden a acciones del jugador, modifican el estado del mundo, y disparan efectos narrativos. Para ello, dispone de un conjunto de variables de sistema y convenciones que permiten controlar la lógica del juego.

3.1. gameState: Estado Dinámico del Juego

El objeto `gameState` representa el estado vivo del universo jugable. Es accesible desde los scripts y contiene toda la información relevante para tomar decisiones narrativas.

Estructura principal

```

gameState = {
  currentPlayer: 'Pancho',
  currentScene: 'quincho',
  actionState: {
    verb: 'USAR',

```

```
    item: { key: 'encendedor', type: 'item' },
    target: { key: 'parrilla', type: 'object' }
  },
  activeItem: 'encendedor',
  inventories: {
    Pancho: ['encendedor', 'vaso_fernet']
  },
  hoverTarget: 'parrilla',
  dialogue: {
    active: false,
    currentLine: null
  },
  lastInteractionTime: 1693920000000
}
```

Campos clave

Variable	Descripción
currentPlayer	ID del personaje que controla el jugador.
currentScene	ID de la escena actual.
actionState	Verbo y argumentos seleccionados por el jugador.
activeItem	Ítem actualmente seleccionado (si aplica).
inventories	Inventario por personaje.
hoverTarget	Elemento bajo el cursor (objeto, ítem o personaje).
dialogue	Estado del diálogo activo.
lastInteractionTime	Timestamp de la última acción ejecutada.

El script puede consultar estas variables para condicionar su ejecución, validar estados, o modificar el mundo.

3.2. Convención de Scripts Narrativos

Los scripts se ejecutan mediante la función `runActionScript(nombre)`, donde el nombre se construye automáticamente según la acción del jugador.

Formato del nombre

`<jugador>_<verbo>_<item>_<target>`

Ejemplo:

`pancho_usar_encendedor_parrilla`

El programador debe definir un script con ese nombre en el objeto `storyScripts`, que contenga la lógica narrativa correspondiente.



3.3. Acceso a Datos del Universo (gameData)

Los scripts pueden acceder a todo el universo definido en `gameData`, incluyendo:

- `gameData.characters`: información de personajes
- `gameData.objects`: estado y propiedades de objetos
- `gameData.items`: ítems disponibles y sus flags (`canBePickedUp`, `isHidden`, etc.)
- `gameData.scenes`: composición de cada escena
- `gameData.dialogueMatrix`: diálogos por personaje

Esto permite que los scripts modifiquen estados, agreguen ítems al inventario, cambien de escena, o disparen diálogos.



3.4. Funciones de Servicio (API narrativa)

Además de las variables de sistema, el programador de scripts tiene acceso a funciones utilitarias que permiten:

- Mostrar texto narrativo
- Modificar inventarios
- Cambiar estados de objetos
- Iniciar diálogos
- Mover personajes
- Cambiar de escena

Estas funciones serán explicadas en el próximo capítulo.



3.5. Filosofía de Diseño

- Los scripts deben ser **modulares**, **autocontenidos** y **deterministas**.
- No deben depender de efectos secundarios no controlados.
- Deben consultar `gameState` y `gameData` para tomar decisiones.
- Pueden modificar el estado del juego, pero deben hacerlo de forma explícita.

El motor no impone restricciones narrativas: el programador tiene libertad total para definir comportamientos, siempre que respete la convención de nombres y estructura.



Capítulo 4: Funciones Narrativas (API para Scripts)

Este capítulo presenta el conjunto de funciones que el programador de la “story” puede invocar desde los scripts narrativos. Estas funciones permiten modificar el estado del juego, controlar el flujo narrativo, manipular inventarios, mover personajes, iniciar diálogos, y más.

Todas las funciones están disponibles globalmente dentro del entorno de ejecución de los scripts, y se espera que se usen de forma modular, explícita y contextual.

4.1. Categorías Funcionales

Las funciones disponibles se agrupan en las siguientes categorías:

Control Narrativo

Función	Propósito
<code>say(text)</code>	Muestra una línea de diálogo en pantalla.
<code>startDialogue(nodeID)</code>	Inicia un diálogo desde un nodo específico.
<code>abortDialogues()</code>	Finaliza cualquier diálogo activo.
<code>wait(frames)</code>	Pausa la ejecución por una cantidad de frames.
<code>setTimer(callback, seconds)</code>	Ejecuta una función luego de un tiempo.
<code>clearTimer(timerID)</code>	Cancela un temporizador activo.

Control de Personajes

Función	Propósito
<code>actorChangeScene(actor, destination)</code>	Mueve un personaje a otra escena.
<code>changePlayer(characterName, ui)</code>	Cambia el personaje controlado por el jugador.
<code>getCurrentPlayer()</code>	Devuelve el ID del jugador actual.

Control de Escena

Función	Propósito
<code>changeScene(sceneName)</code>	Cambia la escena actual.
<code>getCurrentScene()</code>	Devuelve el ID de la escena actual.
<code>moveItemToScene(itemName, sceneName)</code>	Coloca un ítem en una escena.

Inventario

Función	Propósito
<code>addItemToActor(characterName, itemName, ui)</code>	Agrega un ítem al inventario de un personaje.
<code>removeItemFromActor(characterName, itemName, ui)</code>	Quita un ítem del inventario.
<code>getInventory(characterName)</code>	Devuelve el inventario de un personaje.
<code>isItemInActor(characterName, itemName)</code>	Verifica si un personaje tiene un ítem.

Función	Propósito
<code>getItem(itemName)</code>	Devuelve el objeto ítem por ID.
<code>dropItem(itemName)</code>	Elimina un ítem del inventario.
<code>giveItemTo(itemName, characterName)</code>	Transfiere un ítem a otro personaje.

Estado de Objetos

Función	Propósito
<code>setObjectState(objectName, state, value)</code>	Modifica un flag de estado de un objeto.
<code>getObjectState(objectName, state)</code>	Consulta el estado de un objeto.
<code>isObjectInState(objectName, state, value)</code>	Verifica si un objeto está en cierto estado.

Guardado y Carga

Función	Propósito
<code>saveGame(slot)</code>	Guarda el estado del juego.
<code>loadGame(slot)</code>	Carga un estado guardado.
<code>listSaves()</code>	Lista los slots disponibles.
<code>clearSaves()</code>	Borra todos los guardados.

Interfaz

Función	Propósito
<code>hideUI()</code>	Oculto la interfaz.
<code>showUI()</code>	Muestra la interfaz.

4.2. Convención de Uso

- Todas las funciones deben usarse dentro de scripts definidos en `storyScripts`.
- Se recomienda validar el estado del juego antes de modificarlo.
- Las funciones no devuelven valores narrativos, sino que modifican el entorno.
- Algunas funciones requieren el objeto `ui` como parámetro para actualizar la vista.

4.3. Ejemplo de Script Narrativo

```
storyScripts['pancho_usar_encendedor_parrilla'] = () => {
  if (isItemInActor('Pancho', 'encendedor')) {
    say("Pancho prende la parrilla con el encendedor.");
    setObjectState('parrilla', 'isLit', true);
    removeItemFromActor('Pancho', 'encendedor', ui);
  } else {
```

```
    say("Pancho no tiene el encendedor.");  
  }  
};
```

Este script verifica si Pancho tiene el encendedor, modifica el estado de la parrilla, y actualiza el inventario.

Perfecto, Sebastián. A partir del contenido de `data.js`, armé el nuevo capítulo del manual técnico-curatorial. Este capítulo está dirigido al programador de la “story” y explica cómo se organiza la información dentro del archivo que deberá modificar: tanto el universo jugable como los scripts narrativos que responden a las acciones del jugador.

Capítulo 5: Organización del Archivo Narrativo (`data.js`)

Este capítulo describe la estructura y el propósito del archivo `data.js`, que funciona como el núcleo narrativo del juego. Es el único archivo que el programador de la “story” debe modificar directamente, ya que contiene:

- La definición completa del universo jugable (`gameData`)
- Los scripts narrativos que responden a las acciones del jugador (`storyScripts`)

Este archivo es el equivalente moderno a los *story files* de la Z-Machine y a los módulos de escena en SCUMM. Su diseño permite que la lógica narrativa esté completamente separada del motor gráfico y de la interfaz.

5.1. `gameData`: Universo Jugable

Ya documentado en el Capítulo 2, `gameData` contiene:

- `inits`: configuración inicial del jugador y escena
- `characters`: definición de personajes jugables y no jugables
- `objects`: elementos fijos o interactivos del entorno
- `items`: objetos móviles que pueden ser recogidos
- `scenes`: composición de cada escena (fondo, objetos, ítems, personajes)
- `dialogueMatrix`: sistema de diálogo ramificado por personaje

El programador puede modificar descripciones, agregar nuevos ítems, personajes o escenas, y expandir los diálogos sin alterar el motor.

5.2. `storyScripts`: Lógica Narrativa

El objeto `storyScripts` contiene todos los scripts que se ejecutan cuando el jugador realiza una acción. Cada script es una función que puede consultar y modificar el estado del juego, mostrar texto,

mover personajes, iniciar diálogos, etc.

Convención de nombres

Cada script se nombra siguiendo el patrón:

`<personaje>_<verbo>_<item>_<target>`

Ejemplo:

```
'pancho_usar_encendedor_parrilla': (dependencies) => { ... }
```

Esta convención permite mapear acciones del jugador a scripts específicos sin ambigüedad.

Acceso a dependencias

Los scripts reciben un objeto `dependencies` que contiene:

- `gameState`: estado dinámico del juego
- `gameData`: universo jugable
- `scenes`: acceso directo a las escenas

Esto permite que el script modifique estados, consulte inventarios, o cambie de escena.

Ejemplo de script narrativo

```
'usar_encendedor_en_parrilla': (dependencies) => {  
  const parrilla = dependencies.scenes.quincho.objects.parrilla;  
  if (parrilla.hasCarbon && !parrilla.isLit) {  
    parrilla.isLit = true;  
    return "¡Listo! Fuego prendido.";  
  } else if (!parrilla.hasCarbon) {  
    return "Primero hay que poner el carbón.";  
  } else {  
    return "El fuego ya está prendido.";  
  }  
}
```

Este script responde a la acción de usar el encendedor en la parrilla. Verifica el estado del objeto y devuelve una línea narrativa.

Script por defecto

El script `'default'` se ejecuta cuando no existe un script específico para una acción. Contiene lógica genérica para verbos como MIRAR, AGARRAR, USAR, IR, etc.

```
'default': () => {  
  const as = gameState.actionState;  
  switch (as.verb) {
```

```
case "MIRAR":
    say(runActionScript("generic_description"));
    break;
case "AGARRAR":
    say("No puedo agarrar eso.");
    break;
...
}
```

Este script actúa como fallback narrativo y puede ser expandido para cubrir casos comunes.



5.3. Buenas prácticas

- Mantener los scripts cortos y autocontenidos.
 - Usar funciones de la API narrativa para modificar el estado del juego.
 - Evitar lógica compleja dentro de los scripts: delegar en funciones externas si es necesario.
 - Usar `say()` para mostrar texto, y `runActionScript()` para invocar otros scripts.
 - Validar estados antes de modificar objetos o inventarios.
-

¿Querés que prepare una plantilla base para que el programador pueda escribir nuevos scripts fácilmente? También puedo armar un índice de scripts existentes para facilitar la navegación.